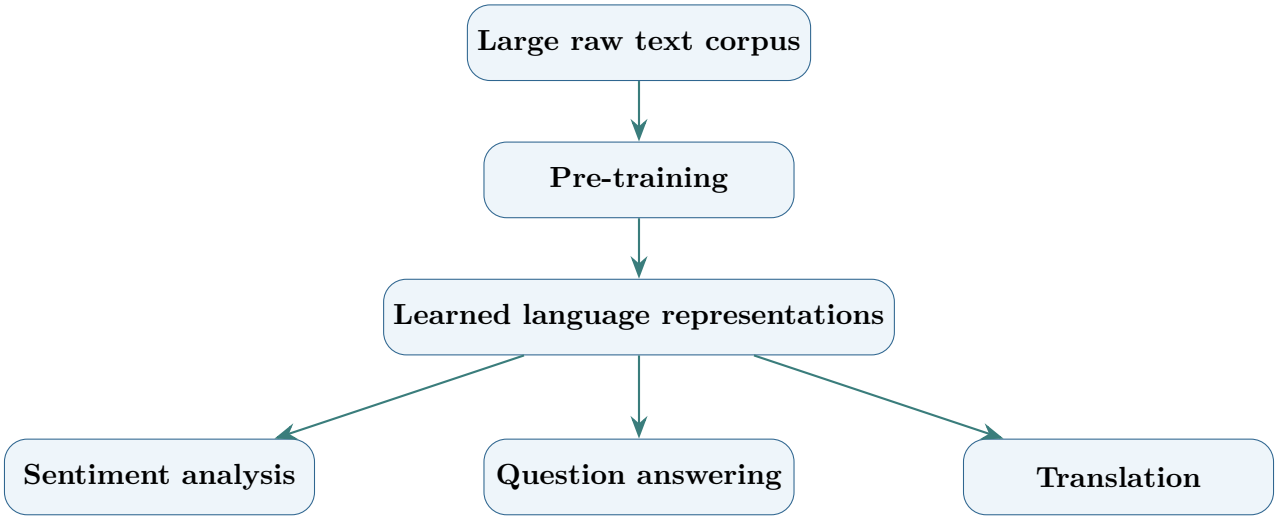
Transfer Learning, Transformers & BERT

Structured NLP Study Notes



Central idea

A model can first learn useful language patterns from large amounts of raw text, then reuse that knowledge for a smaller, task-specific dataset. This is called **transfer learning**.

A study guide with explanations, comparison tables, equations, and visual summaries

Contents

1 Transfer Learning: Reusing What a Model Has Learned **2**
1.1 The overall workflow . . . 2

2 Challenge 1: What Should the Model Learn? **2**
2.1 Possible training objectives . . . 2

3 Challenge 2: How Do We Transfer What the Model Learns? **2**

4 Semi-Supervised Learning and Fine-Tuning **3**
4.1 Question-answering example . . . 3

5 Why Choose the Transformer? **3**
5.1 A. Transformers handle long-range dependencies . . . 3
5.2 B. The same model can handle different NLP tasks . . . 4
5.3 C. Task-specific input and output adaptations . . . 4

6 Zero-Shot Behavior **4**

7 Pre-Training as a Regularization Scheme **4**

8 BERT's Key Idea: Masked Language Modeling **5**

9 Auxiliary Training Objectives **5**

10 Earlier Approaches vs. BERT **6**
10.1 Earlier left-to-right language-model objective . . . 6

11 Training Mechanics: Epochs, Batches, and Optimizer Steps **6**

12 Adam and a Zero Gradient for One Token **7**
12.1 Question . . . 7
12.2 What Adam maintains . . . 7
12.3 What needs correction? . . . 7

13 Positional Encoding Approaches **7**

14 Quick Revision **8**

1 Transfer Learning: Reusing What a Model Has Learned

When you subsequently train a model for sentiment analysis, translation, or another task, it can start with previously learned knowledge instead of learning everything from scratch.

Definition

Transfer learning means reusing knowledge learned from one training stage or task to help a model perform another task.

1.1 The overall workflow

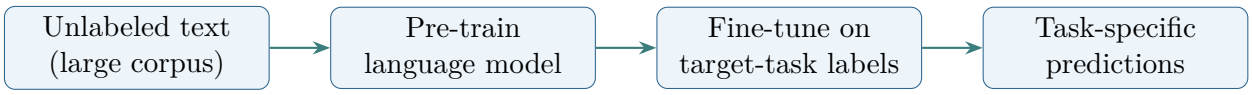


Figure 1: A common transfer-learning workflow.

2 Challenge 1: What Should the Model Learn?

When training a model on raw text, we need an **objective**: a task that tells the model how to learn. The key question is which objective produces representations that are useful for many downstream tasks.

2.1 Possible training objectives

Objective	What the model is trained to do	Example
Language modeling	Predict the next word or token.	"The cat is drinking _____." The model may predict "milk" or another plausible continuation.
Machine translation	Translate a sentence from one language to another.	"Good morning" → "Buenos días". The model learns linguistic patterns useful for translation.
Discourse coherence	Judge whether sentences fit together logically.	Decide whether the second sentence makes sense after the first sentence.

It is not obvious which objective produces the most useful representations across different tasks. An objective might work very well for translation but less well for sentiment analysis. Researchers therefore compare training objectives experimentally.

3 Challenge 2: How Do We Transfer What the Model Learns?

Suppose a model has been pretrained on a huge amount of raw text and is now needed for question answering. How should it be adapted?

Approach	Simple explanation
Change the architecture	Modify the model's structure to suit the target task.
Use intricate learning schemes	Carefully design the training process to adapt the model.
Add auxiliary objectives	Train the model on additional tasks that help it learn useful features.

For example, one could pretrain a language model and then attach a classification layer to perform sentiment analysis.

4 Semi-Supervised Learning and Fine-Tuning

The general idea is to combine:

- **Unlabeled data:** large amounts of raw text that are readily available.
- **Labeled data:** a smaller collection of examples annotated by humans.

For instance, a model could be pretrained on millions of unlabeled sentences and then fine-tuned on just 1,000 labeled sentiment examples. The hope is that the model's prior knowledge reduces its dependence on manually labeled data.

Strictly speaking, pretraining on unlabeled data followed by supervised fine-tuning is often described as **self-supervised pretraining followed by supervised learning**.

4.1 Question-answering example

1. Expose the model to millions of sentences so it can learn language patterns.
2. Train it on examples of questions and their correct answers.
3. Use its previously learned knowledge to perform question answering.

Why is this useful?

You do not need to teach the model all of language from scratch using labeled examples for every task.

The unlabeled pretraining text and labeled target-task data do not need to come from the same domain. For example, a model pretrained on general books and articles might later be fine-tuned on medical questions, although success depends on the domain and task.

5 Why Choose the Transformer?

The authors choose the Transformer architecture instead of relying on older architectures such as recurrent neural networks (RNNs).

5.1 A. Transformers handle long-range dependencies

Consider the sentence:

> *The book that I borrowed from my friend last week was extremely interesting.*

To understand it, the model must connect "book" with "was extremely interesting," even though many words appear between them.

Transformers use **self-attention** to directly model relationships between tokens, helping them capture long-range dependencies. RNNs process tokens sequentially, which can make learning long-range relationships more difficult and limit parallelization during training.

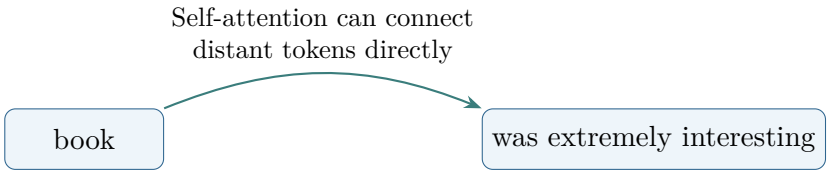


Figure 2: Illustration of a long-range relationship between tokens.

5.2 B. The same model can handle different NLP tasks

Tasks mentioned in the passage include:

- Machine translation
- Document generation
- Syntactic parsing

These tasks have different outputs, but they all involve understanding or generating language. The Transformer provides a flexible architecture that can learn useful representations across them.

5.3 C. Task-specific input and output adaptations

The paper describes making minimal architectural changes when adapting the pretrained model to a particular task.

6 Zero-Shot Behavior

Zero-shot evaluation generally means asking the model to perform a task without training it on labeled examples specifically for that task.

For example, after pretraining, researchers might test whether the model can distinguish meaningful linguistic relationships before fine-tuning it on a dedicated inference dataset. This helps investigate whether pretraining itself teaches useful linguistic knowledge, rather than attributing every improvement to supervised fine-tuning.

In this paper, zero-shot analysis is evidence about what the pretrained representations have learned. It does **not** mean BERT performs every downstream task perfectly without fine-tuning.

7 Pre-Training as a Regularization Scheme

Regularization is a broad concept that helps prevent a model from fitting training data in a way that generalizes poorly.

In this context, pre-training provides a useful starting point: the model has already learned structure from a large corpus before seeing a smaller task-specific dataset. Consequently, fine-tuning may generalize better than training from scratch, although this is not guaranteed in every setting.

The passage notes that this idea had already been explored in:

- Image classification
- Speech recognition
- Entity disambiguation

- Machine translation

Historical distinction

BERT did **not** invent pre-training itself. Its contribution is associated with how it applies bidirectional contextual pre-training using a Transformer encoder.

8 BERT's Key Idea: Masked Language Modeling

BERT uses a Transformer encoder that can learn from both left and right context during pre-training through **masked language modeling (MLM)**.

Example:

> The man went to the `[MASK]` to deposit his money.

The model predicts the masked word using the surrounding context, including the words that follow it.

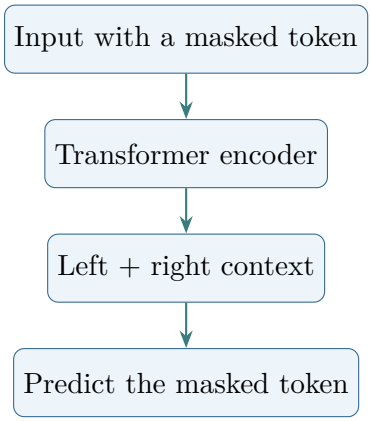


Figure 3: Simplified masked language modeling workflow.

9 Auxiliary Training Objectives

Researchers have also tried to improve NLP models by training them with additional objectives alongside the main task. A multi-objective training setup might minimize:

$$\boxed{L_{\text{total}} = L_{\text{main}} + \lambda L_{\text{aux}}}$$

where:

- L_{main} measures errors on the main task.
- L_{aux} measures errors on the additional task.
- λ controls the relative importance of the auxiliary objective.

The argument in the passage is that pre-training can already teach a model several linguistic properties that help downstream tasks, so it may not need a complicated collection of additional objectives during task-specific training.

BERT still uses a pre-training objective—masked language modeling in the original paper. The authors are not saying auxiliary objectives are useless. Their argument is that this pre-training approach can learn useful linguistic information without requiring a separate collection of auxiliary tasks for every target task.

10 Earlier Approaches vs. BERT

Approach	What it learns or does	Limitation highlighted in the passage
Word-level statistics and embeddings	Word frequencies, associations, and representations.	Often limited in contextual and sentence-level information.
Earlier language-model pre-training	Learns parameters from unlabeled text before fine-tuning.	Conventional left-to-right pre-training has limited access to future context.
Auxiliary objectives	Adds extra learning tasks to improve representations.	Requires choosing and combining additional objectives.
Task-specific feature approaches	Uses representations from a separately pretrained model as features.	Can require additional parameters and architecture changes.
BERT	Learns bidirectional contextual representations through masked language modeling.	Still requires task-specific fine-tuning and has its own limitations.

10.1 Earlier left-to-right language-model objective

The passage gives the objective:

$$L_1 = \sum_i \log P(u_i \mid \text{previous tokens}; \Theta)$$

The training objective is to **maximize** L_1.

11 Training Mechanics: Epochs, Batches, and Optimizer Steps

Suppose a dataset contains 1,000 examples and the batch size is 100:

- One epoch contains $1000/100 = 10$ batches.
- Each batch usually triggers one optimizer step.
- Therefore, one epoch contains 10 optimizer updates, assuming standard mini-batch training.

  If the batch size is 1, then one epoch over 1,000 examples typically contains 1,000 optimizer steps.

Remember

Parameters are updated once per optimizer step, not necessarily only once per epoch.

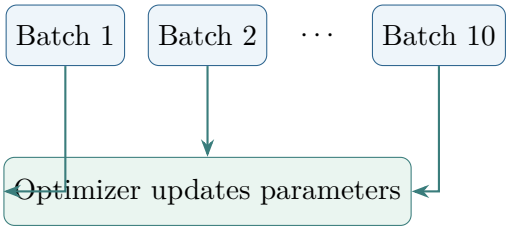


Figure 4: With 1,000 examples and batch size 100, a standard epoch has 10 batches and typically 10 optimizer steps.

12 Adam and a Zero Gradient for One Token

12.1 Question

Suppose you use the Adam optimizer and token 2's accumulated gradient in one batch is zero:

$$\nabla E_2 = 0$$

Can token 2's embedding still change after the Adam update, or is it guaranteed to remain unchanged? Consider that Adam also has gradient history from previous batches.

12.2 What Adam maintains

Adam maintains two moving averages for each parameter:

- **First moment:** a moving average of gradients.
- **Second moment:** a moving average of squared gradients.

  This history lets Adam use adaptive updates and can help optimization.

Answer to the question

A zero gradient in the current batch does **not necessarily guarantee** that the embedding remains unchanged. The first-moment estimate may still contain information from previous non-zero gradients, so the update can remain non-zero.

12.3 What needs correction?

The claim that Adam therefore eliminates vanishing and exploding gradients is not fully correct.

- Adam does not guarantee that vanishing gradients are solved.
- Adam does not automatically eliminate exploding gradients.
- Techniques such as gradient clipping may still be needed.
- Adam's main benefits include adaptive updates and use of gradient history.

Adam can help optimization, but it is not a universal cure for every gradient-related problem.

13 Positional Encoding Approaches

Approach	Implementation difficulty	What you learn
Learned positional embeddings	Easy	Parameters, gradient accumulation, and backpropagation.
Sinusoidal encoding	Easy–medium	Fixed positional functions; no learned positional parameters.
RoPE	Medium–hard	Position-aware attention and relative-position behavior.

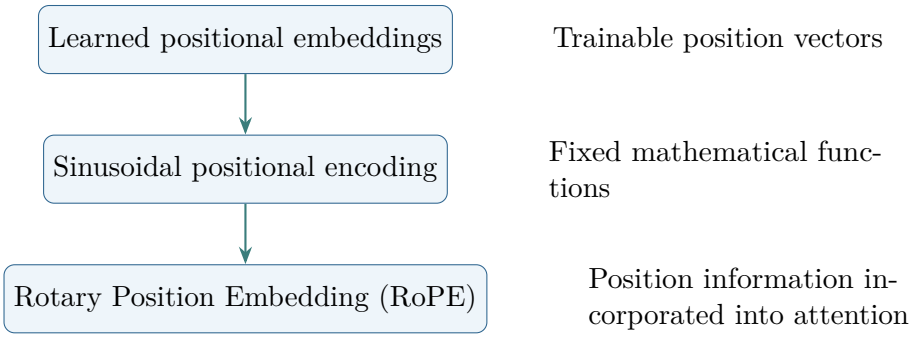


Figure 5: A high-level visual comparison of the positional approaches listed in the notes.

14 Quick Revision

- **Transfer learning:** reuse previously learned knowledge for a new task.
- **Pre-training:** learn useful patterns from a large corpus before task-specific training.
- **Fine-tuning:** adapt a pretrained model to a target task, often with labeled examples.
- **Self-supervised pretraining:** create a learning signal from the data itself, such as predicting masked tokens.
- **Transformer self-attention:** helps relate tokens even when they are far apart in a sequence.
- **Zero-shot evaluation:** test a task without task-specific labeled training examples.
- **BERT's key idea in these notes:** bidirectional contextual representations through masked language modeling.
- **Auxiliary loss:** an additional training objective combined with the main loss.
- **Epoch vs. optimizer step:** an epoch may contain many batches and therefore many parameter updates.
- **Adam:** uses moving averages of gradients and squared gradients; it does not guarantee elimination of vanishing or exploding gradients.